

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	U.Udhayashree	Reg. No.:	192521448
Section / Batch:	2025-Btech-IT	Date:	28/07/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	A university system maintains student records in an unsorted list due to frequent updates and uses Linear Search for retrieval. a) Explain how Linear Search works in this scenario. b) Analyze best, average, and worst-case performance. c) Justify whether sorting would improve efficiency.	Q1

SECTION A – SCENARIO BASED ASSESSMENT

A financial system uses Binary Search for sorted transaction records, but frequent updates disturb order. a) Explain Binary Search operation. b) Analyze the impact of unsorted data. c) Discuss maintaining sorted data for efficiency.

Code of Conduct

I U.Udhayashree-192521448 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: U.Udhayashree

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex realworld problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & DecisionMaking Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				
Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

A university system maintains student records in an unsorted list due to frequent updates and uses Linear Search for retrieval. a) Explain how Linear Search works in this scenario. b) Analyze best, average, and worst-case performance. c) Justify whether sorting would improve efficiency.

a) Explain how Linear Search works in this scenario

A university maintains student records in an **unsorted list** because new records are frequently added, updated, and deleted. To find a student's record, the system uses **Linear Search**. The search starts from the first record and compares each student's **ID, roll number, or name** with the required search key. If a match is found, the search stops and displays the student's details. If no match is found, the algorithm continues checking each record until the end of the list. If all records are checked and no match is found, the system reports that the student record is not available.

Steps:

- Start from the first student record.
- Compare the search key with the current record.
- If it matches, return the student's details.
- Otherwise, move to the next record.
- Repeat until the record is found or the list ends.
- If not found, display "**Record Not Found.**"

b) Analyze Best, Average, and Worst-Case Performance

- **Best Case – $O(1)$:** The required student record is the first record, so only one comparison is needed.
- **Average Case – $O(n)$:** The record is found somewhere in the middle, so about half of the records are checked on average.
- **Worst Case – $O(n)$:** The required record is the last one or does not exist, so every record in the list must be checked.

Advantages of Linear Search:

- Easy to understand and implement.
- Works well with unsorted data.
- No need to sort the records.
- Suitable for frequent updates.

c) Justify whether sorting would improve efficiency

Sorting the records would improve search efficiency because Binary Search could be used, which works in $O(\log n)$ time. However, in a university system, student records are updated frequently. Every insertion, deletion, or modification would require the records to be sorted again, increasing the maintenance cost.

Therefore, if updates are more frequent than searches, keeping the records unsorted and using Linear Search is the better choice. If searches are performed much more frequently than updates, then sorting the records and using Binary Search would provide better performance.

Conclusion:

For a university system with frequent record updates, Linear Search on an unsorted list is a practical and efficient solution because it avoids the overhead of maintaining a sorted list while still providing simple and reliable searching.

Program:

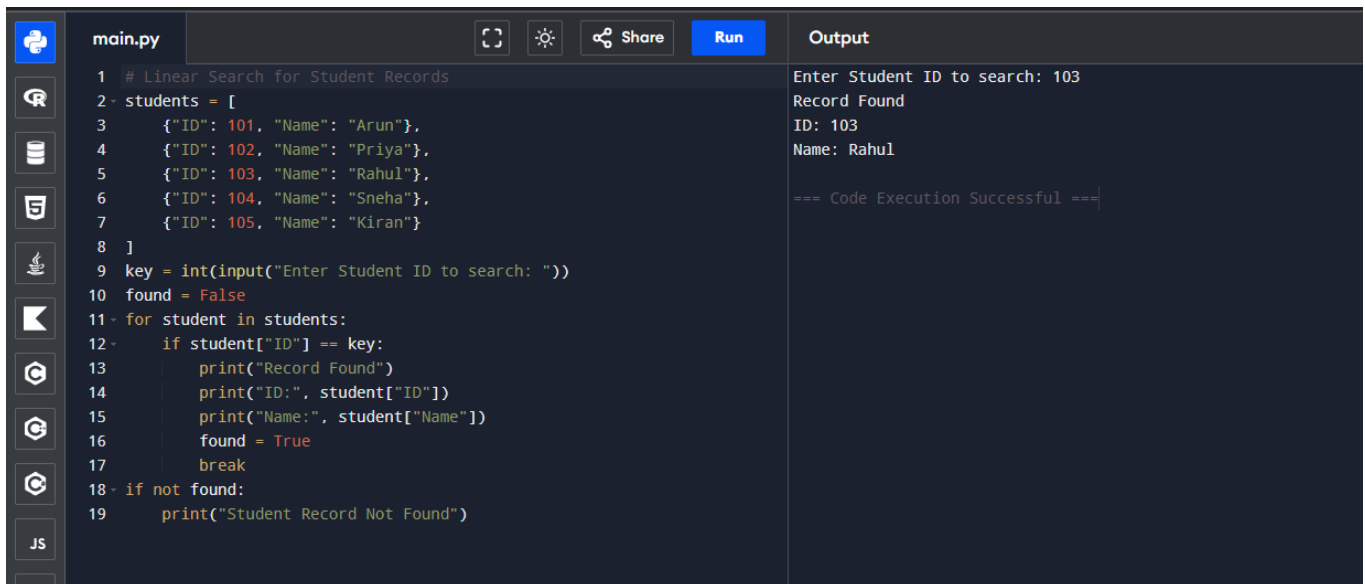
Linear Search for Student Records

```
students = [  
    {"ID": 101, "Name": "Arun"},  
    {"ID": 102, "Name": "Priya"},  
    {"ID": 103, "Name": "Rahul"},  
    {"ID": 104, "Name": "Sneha"},  
    {"ID": 105, "Name": "Kiran"}  
]  
  
key = int(input("Enter Student ID to search: "))  
  
found = False  
  
for student in students:
```

```
if student["ID"] == key:
    print("Record Found")
    print("ID:", student["ID"])
    print("Name:", student["Name"])
    found = True
    break
```

if not found:

```
print("Student Record Not Found")
```



```
main.py
1 # Linear Search for Student Records
2 students = [
3     {"ID": 101, "Name": "Arun"},
4     {"ID": 102, "Name": "Priya"},
5     {"ID": 103, "Name": "Rahul"},
6     {"ID": 104, "Name": "Sneha"},
7     {"ID": 105, "Name": "Kiran"}
8 ]
9 key = int(input("Enter Student ID to search: "))
10 found = False
11 for student in students:
12     if student["ID"] == key:
13         print("Record Found")
14         print("ID:", student["ID"])
15         print("Name:", student["Name"])
16         found = True
17         break
18 if not found:
19     print("Student Record Not Found")
```

Output

```
Enter Student ID to search: 103
Record Found
ID: 103
Name: Rahul

=== Code Execution Successful ===
```